SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4307
STUDENT NAME: K. Ganesh
REGISTER NUMBER: 192411170

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Code along exercise: Make your first HTML page together with CSS

Weightage: 10%

QUESTIONS: **Total Marks: 10**

Scenario: Web Development Standards Implementation

A start up is developing a modern web application. During development, the team encounters multiple issues related to HTML elements, HTTP methods, XML syntax, and web communication protocols.
The following observations are made:

- The team needs to embed a promotional video on the homepage.
- A registration form must securely send user data to the server.
- The navigation menu must follow semantic HTML standards.
- XML configuration files are being used but contain syntax errors.
- The application must communicate over the correct web protocol.

Data Snippets Provided

Snippet 1: Video Embedding Options

```
A. <media src="video.mp4"></media>
B. <video src="video.mp4" controls></video>
C. <movie src="video.mp4"></movie>
D. <vid src="video.mp4"></vid>
```

Snippet 2: Form Submission Methods

```
A. GET
B. POST
C. PUT
D. DELETE
```

Snippet 3: Navigation Elements

```
A. <div>
B. <nav>
C. <section>
D. <header>
```

Snippet 4: XML Syntax

```
A. <note><to>User</to><from>Admin</from>
B. <note><to>User</to><from>Admin</from></note>
C. <note><to>User</from></note>
D. <note to="User"></note
```

Snippet 5: Web Communication Protocols

```
A. FTP
B. HTTP
C. SMTP
D. SNMP
```

Questions:

1. Select the **correct HTML tag for video embedding** and justify your answer.
2. Identify the **appropriate HTTP method for form submission** and explain why.
3. Choose the **proper semantic element for navigation** and justify its usage.
4. Identify the **correct XML syntax** and explain the error in other options.
5. Choose the **correct protocol for web communication** and justify your choice.

Code of Conduct:

I K. Ganesh / 192411170 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *K. Ganesh*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Understanding

The start-up's web application has five separate problems: embedding a promotional video, sending registration data securely, building a navigation menu with semantic HTML, fixing XML configuration files that contain syntax errors, and choosing the right web protocol. For each problem below I picked the correct option from the snippet, justified it, and explained why the other options do not work.

Question 1: Video embedding

Select the correct HTML tag for video embedding and justify your answer.

Answer: Option B - **`<video src="video.mp4" controls></video>`**.

HTML5 added the **`<video>`** element as the standard way to play video on a page, and it works without any plugin. The **`src`** attribute points to the file and the **`controls`** attribute shows the browser's own play, pause, seek, volume and fullscreen buttons. Without **`controls`** the visitor would see a still frame and have no way to start the video. Options A (**`<media>`**), C (**`<movie>`**) and D (**`<vid>`**) are not elements in the HTML specification. Browsers simply ignore unknown tags, so nothing would play, and the page would also fail validation.

For a good production page I would also add **`width`**, **`poster`** (a preview image), a fallback sentence inside the tag, and a second **`<source>`** in WebM format for wider browser support.

Snippet 1: video embedding options

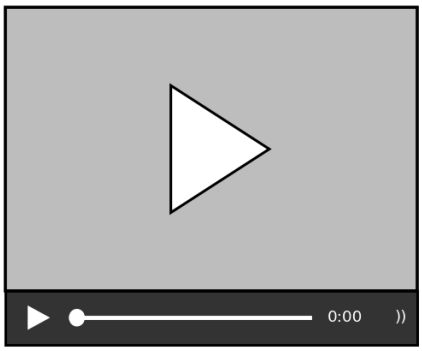
Ⓐ `<media src="video.mp4"></media>`
Invalid: no such element

Ⓑ `<video src="video.mp4" controls></video>`
Correct: HTML5 video element

Ⓒ `<movie src="video.mp4"></movie>`
Invalid: no such element

Ⓓ `<vid src="video.mp4"></vid>`
Invalid: no such element

Output of option B



controls bar: play, seek, volume

Figure 1: Video tag options and the player the browser shows for option B

Conclusion: Option B is the only valid element. It gives a working, accessible player with one line of HTML.

Question 2: Form submission method

Identify the appropriate HTTP method for form submission and explain why.

Answer: Option B, POST.

A registration form sends private data (name, email, password) and asks the server to create something new, an account. POST fits this because the data travels inside the request body and not in the URL. So it does not show up in the address bar, browser history, bookmarks or normal access logs, it has no practical size limit, and it is not cached.

- **GET:** puts the values in the URL (for example **`/register?password=1234`**), which exposes them. GET is meant only for fetching data.
- **PUT:** replaces or updates a resource that already exists at a known address. It is not for creating a new sign-up, and plain HTML forms cannot send it.
- **DELETE:** removes a resource, so it is the wrong action for registration. HTML forms cannot send it either.

POST alone does not encrypt anything. The form must be served over HTTPS, and the server should hash the password, validate the input and use a CSRF token.

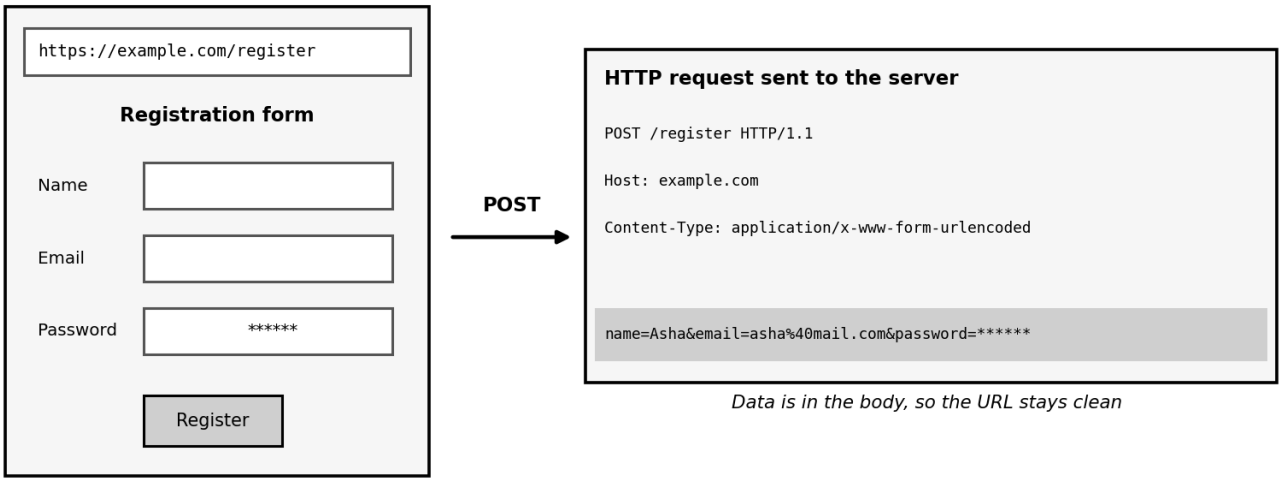


Figure 2: Registration form sending its data with POST

> **Conclusion:** POST is the correct and safest choice for the registration form, as long as it is used together with HTTPS.

Question 3: Semantic navigation

Choose the proper semantic element for navigation and justify its usage.

Answer: Option B - **`<nav>`**.

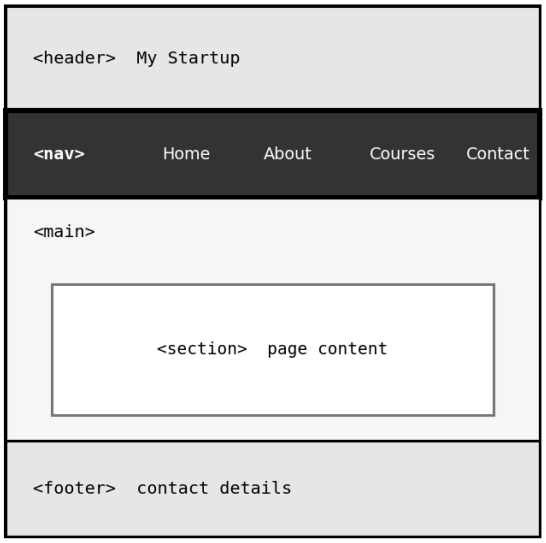
The **`<nav>`** element is made for the main block of links that moves people around a site. Because the tag itself carries the meaning, browsers, search engines and screen readers all know that this block is the navigation menu. A screen-reader user can jump straight to it, search engines understand the site structure better, and the code is easier for the team to read and style.

- **`<div>`:** a generic container with no meaning. It would need extra roles or class names to say "this is a menu".
- **`<section>`:** groups related content under a heading, such as an article section. A set of links is not that.
- **`<header>`:** holds the introductory part of a page (logo, title). It can contain a **`<nav>`**, but it is not the navigation itself.

Basic CSS to turn the list of links into a horizontal menu:

```
nav ul { list-style: none; display: flex; gap: 20px;
         margin: 0; padding: 12px; background: #222; }
nav a  { color: #fff; text-decoration: none; }
```

Page layout (output)



Matching markup

```
<header>
  <h1>My Startup</h1>
  <nav>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="about.html">About</a></li>
      <li><a href="courses.html">Courses</a></li>
      <li><a href="contact.html">Contact</a></li>
    </ul>
  </nav>
</header>
```

Figure 3: Where the nav element sits in the page and the markup that creates it

> **Conclusion:** Using **<nav>** keeps the page semantic, accessible and search-friendly, which a plain div or section cannot do.

Question 4: XML syntax

Identify the correct XML syntax and explain the error in other options.

Answer: Option B - **`<note><to>User</to><from>Admin</from></note>`**.

XML is strict. A document is well-formed only when it has one root element, every start tag has a matching end tag, tags are nested in the right order, and attribute values are quoted. Option B follows all of these rules. The errors in the other options are:

- **Option A:** the root element **`<note>`** is opened but never closed, because the final **`</note>`** is missing.
- **Option C:** **`<to>`** is closed with **`</from>`**. The tag names do not match, and **`<from>`** was never opened, so the nesting is broken.
- **Option D:** the end tag **`</note`** has no closing **`>`**. The attribute **`to="User"`** itself is fine, but the tag is not properly finished.

Unlike HTML, an XML parser does not repair mistakes. It stops at the first error, so a broken configuration file can stop the whole application from starting.

Snippet 4: which XML is well-formed?

A	`<note><to>User</to><from>Admin</from>`	Not valid: root <note> is never closed *(no </note> at the end)*
B	`<note><to>User</to><from>Admin</from></note>`	**Well-formed: one root, all tags closed** *and nested in the right order*
C	`<note><to>User</from></note>`	Not valid: <to> is closed by </from> *(mismatched tags, <from> never opened)*
D	`<note to="User"></note`	Not valid: end tag is incomplete *(the closing > is missing)*

Figure 4: XML options with the reason each one is valid or invalid

Conclusion: Only option B is well-formed XML. Checking every file with an XML validator before deployment would catch the other three kinds of error early.

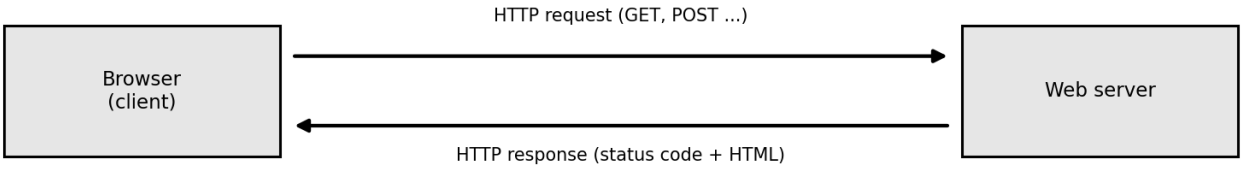
Question 5: Web communication protocol

Choose the correct protocol for web communication and justify your choice.

Answer: Option B, HTTP.

HTTP (HyperText Transfer Protocol) is the application-layer protocol of the web. The browser sends a request such as GET or POST, and the server replies with a status code and the content, usually HTML. It normally runs over TCP on port 80. Since the start-up's site collects passwords, it should use HTTPS, which is HTTP carried over TLS encryption on port 443.

- **FTP:** transfers files between computers. It is not used to load web pages, and it sends credentials in plain text.
- **SMTP:** sends email between mail servers, so it has nothing to do with showing web pages.
- **SNMP:** monitors and manages network devices such as routers and switches.



Protocol	Full form	Purpose	Default port
FTP	File Transfer Protocol	Moving files between computers	21
HTTP	**HyperText Transfer Protocol**	**Requesting and delivering web pages**	**80 (HTTPS: 443)**
SMTP	Simple Mail Transfer Protocol	Sending email between mail servers	25 / 587
SNMP	Simple Network Management Protocol	Monitoring network devices	161

Highlighted row: the protocol the web uses (use the HTTPS version in production)

Figure 5: HTTP request and response, compared with the other protocols

Conclusion: HTTP, secured as HTTPS, is the protocol built for browser and web server communication, and the other three serve different purposes.

Summary of Solutions

Observation	Choice	Main reason
Embed promotional video	B: <video ... controls>	The only real HTML5 video element; controls give a player
Secure registration form	B: POST	Data goes in the body, not in the URL
Semantic navigation menu	B: <nav>	Meaning for browsers, screen readers and search engines
XML configuration	B: complete note element	One root, all tags closed and nested correctly
Web communication	B: HTTP (HTTPS)	Protocol designed for browser-server requests and responses